

Digital Design and Computer Organisation Laboratory

UE23CS251A

3rd Semester, Academic Year 2024-25

DDCO MINI PROJECT

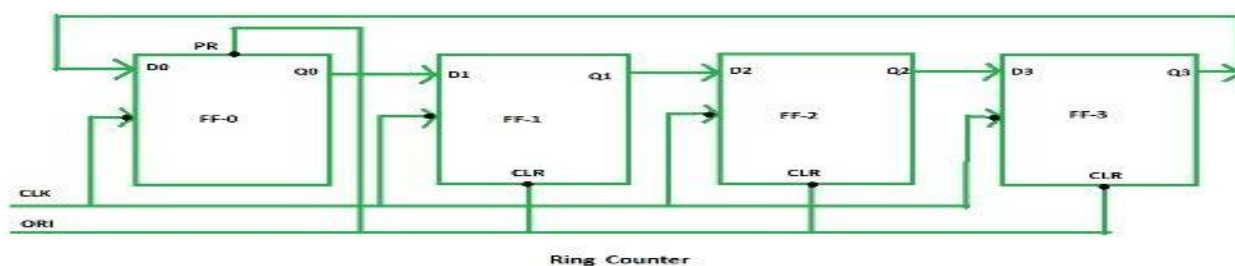
Section-K: Team-9

Date:20/11/2024

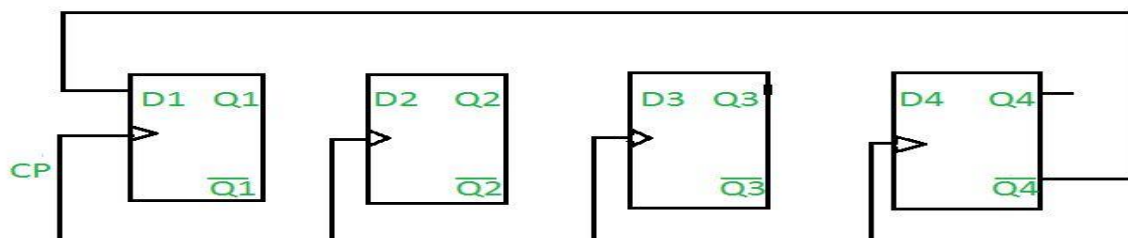
Name:	SRN:	Section
Dipak Rajendra Chaudhari	PES1UG24CS804	K
Vaibhav Srivastava	PES1UG23CS666	K
Tejas Anil Nagmote	PES1UG23CS646	K
Utkarsh Vats	PES1UG23CS661	K

Problem Statement: Design& implement a Ring and Johnson Counter with control logic.

Ring counter:-



Johnson Counter:-



ring_counter.v:-

```
module ring_counter (  
    input wire clk,    // Clock signal  
    input wire rst_n,  // Active-low reset signal  
    output reg [3:0] q // 4-bit output for Ring Counter  
);  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            q <= 4'b1000; // On reset, initialize the counter to "1000" with one '1'  
        else  
            q <= {q[2:0], q[3]}; // Circular left shift (rotate)  
        end  
    endmodule
```

johnson_counter.v:-

```
module johnson_counter (  
    input clk,  
    input rst_n,    // Active-low reset  
    output reg [3:0] q // 4-bit output  
);  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            q <= 4'b0000; // Initialize to zero on reset  
        else  
            q <= {~q[3], q[3:1]}; // Shift left and feed complement of MSB into LSB  
        end  
    endmodule
```

counter_control.v:-

```
module counter_control(  
    input clk,  
    input rst_n,    // Active-low reset  
    input mode,      // Mode selection: 0 for Ring Counter, 1 for Johnson Counter  
    output reg [3:0] q // 4-bit output  
);  
    // Internal registers to store the state of the counters  
    reg [3:0] ring_counter;  
    reg [3:0] johnson_counter;  
  
    always @(posedge clk or negedge rst_n) begin
```

```

if (!rst_n) begin
    // Reset both counters to zero
    ring_counter <= 4'b0001; // Set initial state for Ring Counter to avoid undefined state
    johnson_counter <= 4'b0000; // Reset Johnson Counter
end else begin
    if (mode == 0) begin
        // Ring Counter Logic: Shift left, MSB is fed back to LSB
        ring_counter <= {ring_counter[2:0], ring_counter[3]};
        q <= ring_counter; // Output the ring counter
    end else begin
        // Johnson Counter Logic: Complement MSB and feed it to LSB
        johnson_counter[3] <= ~johnson_counter[0];
        johnson_counter[2:0] <= johnson_counter[3:1];
        q <= johnson_counter; // Output the johnson counter
    end
end
end
endmodule

```

tb_counter_control.v:-

```

module tb_counter_control;

    reg clk;
    reg rst_n;    // Active-low reset
    reg mode;     // Mode selection: 0 for Ring Counter, 1 for Johnson Counter
    wire [3:0] q; // 4-bit output from the counter control module

    // Instantiate the counter control module (uut = Unit Under Test)
    counter_control uut (
        .clk(clk),
        .rst_n(rst_n), // Active-low reset
        .mode(mode),
        .q(q)
    );

    // Generate clock signal with a period of 10 time units (frequency = 100 MHz)
    always #5 clk = ~clk;

    // Testbench stimulus
    initial begin
        // Initialize inputs
        clk = 0;    // Clock starts at 0
        rst_n = 0;  // Assert reset (active-low reset)
        mode = 0;   // Start with Ring Counter mode

        // Apply reset for 10 time units
        #10 rst_n = 1; // De-assert reset (active-low reset)
        // Observe Ring Counter behavior for 50 time units
    end
endmodule

```

```

#50 mode = 1; // Switch to Johnson Counter after 50 time units
// Observe Johnson Counter behavior for 200 time units
#200 $finish; // End simulation after 200 time units
end

// Monitor outputs: Print the mode and output 'q' values at each change
initial begin
    $monitor("Time = %0t | mode = %b | q = %b", $time, mode, q);
end

// Dump waveforms for debugging: This creates a .vcd file for waveform viewing in a viewer like
GTKWave
initial begin
    $dumpfile("dumpfile.vcd");
    $dumpvars(0, tb_counter_control);
end
endmodule

```

Outputs:-

Console Output:-

```

C:\DDCO_IVERILOG\project>vvp output_file
VCD info: dumpfile dumpfile.vcd opened for output.
Time = 0 | mode = 0 | q = xxxx
Time = 15 | mode = 0 | q = 0001
Time = 25 | mode = 0 | q = 0010
Time = 35 | mode = 0 | q = 0100
Time = 45 | mode = 0 | q = 1000
Time = 55 | mode = 0 | q = 0001
Time = 60 | mode = 1 | q = 0001
Time = 65 | mode = 1 | q = 0000
Time = 75 | mode = 1 | q = 1000
Time = 85 | mode = 1 | q = 1100
Time = 95 | mode = 1 | q = 1110
Time = 105 | mode = 1 | q = 1111
Time = 115 | mode = 1 | q = 0111
Time = 125 | mode = 1 | q = 0011
Time = 135 | mode = 1 | q = 0001
Time = 145 | mode = 1 | q = 0000
Time = 155 | mode = 1 | q = 1000
Time = 165 | mode = 1 | q = 1100
Time = 175 | mode = 1 | q = 1110
Time = 185 | mode = 1 | q = 1111
Time = 195 | mode = 1 | q = 0111
Time = 205 | mode = 1 | q = 0011
Time = 215 | mode = 1 | q = 0001
Time = 225 | mode = 1 | q = 0000
Time = 235 | mode = 1 | q = 1000
Time = 245 | mode = 1 | q = 1100
Time = 255 | mode = 1 | q = 1110
tb_counter_control.v:33: $finish called at 260 (1s)

```

Gtkwave Output:-

